

ManteStock — Documento técnico del proyecto

Francisco Javier Garvito Rojas Óscar Arévalo Huertas
Ever Filadelfo Melo Delgado Edison Yesid Quintero Castro
Claudia Marcela Villamil Huertas

25 de julio de 2026

Índice general

1	Portada e información general	3
2	Planteamiento del problema y solución	4
2.1	El problema	4
2.2	La solución	4
2.3	Producto Mínimo Viable (MVP) — matriz MoSCOW	4
2.4	Usuarios del sistema	5
2.5	Historias de usuario principales	5
3	Arquitectura y stack tecnológico	7
3.1	Vista general	7
3.2	Capas y responsabilidades	7
3.3	Justificación del stack	7
3.4	Autenticación y seguridad	9
3.5	Patrón de transacciones	11
4	Modelo de datos	12
4.1	Diagrama entidad-relación	12
4.2	Entidades principales	12
4.3	Sobre createdBy (decisión de diseño relevante)	12
4.4	Discrepancias respecto al modelo original planeado	13
5	Funcionalidades desarrolladas	14
5.1	Catálogo de materiales	14
5.2	Inventario y movimientos	14
5.3	Autenticación y usuarios	14
5.4	Trazabilidad y auditoría	14
5.5	Consulta y dashboard	14
5.6	Pantallas del frontend (8 vistas implementadas)	14
6	Uso de IA como copiloto	16
6.1	Ejemplos concretos de uso	16
6.2	Qué se aceptó, qué se descartó y por qué	17

7	Instrucciones de ejecución local	18
7.1	Requisitos previos	18
7.2	Pasos	18
7.3	Verificación	18
8	Conclusiones y trabajo futuro	20
8.1	Aprendizajes principales	20
8.2	Qué faltó	20
8.3	Cómo se abordaría en una siguiente etapa	20
9	Componente investigativo	21
9.1	Marco de referencia	21
9.2	Referencias	22

1 Portada e información general

Título del proyecto: ManteStock — Sistema de inventario para mantenimiento industrial

Estudiantes: Francisco Javier Garvito Rojas, Óscar Arévalo Huertas, Ever Filadelfo Melo Delgado, Edison Yesid Quintero Castro, Claudia Marcela Villamil Huertas

Seminario: Seminario de Desarrollo de Software

Fecha de entrega: 25 de julio de 2026

Repositorio en GitHub: https://github.com/cmwillamih-sketch/Proyecto_Inventario — rama feature/checkpoints-007-010

Nota sobre el estado del repositorio: el trabajo descrito en este documento vive en la rama feature/checkpoints-007-010, no en main. Esto es intencional: la metodología del curso exige Gitflow (rama de trabajo + Pull Request + revisión de código antes de integrar a main), y esa integración queda pendiente, a decidir en una siguiente etapa. Todo el código, los commits y la documentación citados en este documento están disponibles en esa rama.

2 Planteamiento del problema y solución

2.1 El problema

En muchas áreas de mantenimiento industrial, el control de repuestos e insumos se lleva en hojas de cálculo y mensajes informales (WhatsApp, notas verbales). Esto genera consecuencias conocidas y documentadas en la literatura sobre gestión de inventarios (Durán, 2012): falta de visibilidad sobre el stock real, compras duplicadas por desconocimiento de existencias, errores acumulados en el conteo, y ausencia de trazabilidad cuando se necesita reconstruir qué pasó con un material.

Formulado como una única frase (técnica de “servilleta”):

Un técnico o coordinador de mantenimiento industrial necesita saber con certeza qué stock de repuestos existe y quién movió qué, pero se enfrenta a un control basado en hojas de Excel y mensajes de WhatsApp sin una fuente única de verdad, lo que provoca compras duplicadas, salidas sin registrar y pérdida de trazabilidad ante auditorías.

2.2 La solución

ManteStock centraliza el control de inventario de mantenimiento industrial en una única fuente de verdad, con trazabilidad completa de cada movimiento y control de acceso por rol, reemplazando el registro manual disperso. La administración eficiente del inventario no es un detalle operativo menor: representa una de las inversiones más importantes de una organización en relación con el resto de sus activos, y su mala gestión tiene un impacto financiero directo (Durán, 2012).

2.3 Producto Mínimo Viable (MVP) — matriz MoSCOW

2.3.1 Must (imprescindible — entregado y verificado)

- Catálogo de materiales (alta, baja, edición, consulta, búsqueda por código/descripción).
- Registro de entradas, salidas y ajustes de inventario, con validación de stock y transacción atómica.
- Autenticación con JWT, bloqueo por intentos fallidos, permisos diferenciados por rol.
- Trazabilidad completa: historial por material y por usuario, filtros por fecha/tipo/responsable, registros inmutables.
- Dashboard con indicadores generales y materiales en stock bajo.

2.3.2 Should (importante, quedó pendiente)

- Pantalla de gestión de usuarios en el frontend (crear/editar/resetear contraseña/desactivar) — hoy solo existe por API.
- Soporte de stock en unidades decimales/fraccionadas (kg, litros).
- Reemplazar el JWT_SECRET de desarrollo por un secreto real antes de cualquier despliegue.

2.3.3 Could (deseable si sobra tiempo)

- Bloquear que un administrador se autodegrade de rol (hoy solo está bloqueada la autodesactivación).
- Documentación de API tipo OpenAPI/Swagger.
- Exportar historial de movimientos a PDF/Excel.

2.3.4 Won't (fuera de alcance de esta versión, decisión consciente)

- **Módulo de Notificaciones** (alertas de stock mínimo, movimientos críticos) — descartado explícitamente el 21/07/2026, no es un pendiente.
- Integración con ERP o proveedores externos.
- Pasarelas de pago electrónico.
- Automatización de mensajería (WhatsApp, SMS).
- Operación multi-sucursal, aplicación móvil nativa, analytics avanzados.
- Despliegue en la nube (fuera de alcance de esta cohorte del seminario, según sección 1 de la guía de entrega).

2.4 Usuarios del sistema

Rol	Qué hace en el sistema
Técnico	Consulta stock, registra entradas/salidas/ajustes, gestiona el catálogo de materiales.
Coordinador de compras	Mismo acceso funcional que Técnico — no hay restricciones de rol adicionales sobre Materiales/Inventario en esta versión (decisión de alcance documentada).
Administrador	Todo lo anterior, más gestión de cuentas de usuario (crear, activar/desactivar, resetear contraseña, asignar rol).

2.5 Historias de usuario principales

Formato: *Como [rol], quiero [acción], para [beneficio]*, con criterios de aceptación Dado/Cuando/Entonces. Se listan las 6 más representativas; el listado completo (11 historias) está en HISTORIAS-DE-USUARIO.md del repositorio.

HU01 — Iniciar sesión. Como usuario del sistema, quiero iniciar sesión con usuario y contraseña, para acceder a las funciones según mi rol. - Dado un usuario y contraseña correctos, cuando envío el formulario de login, entonces el sistema me redirige al dashboard con una sesión activa. - Dado que fallo el login 2 veces seguidas, cuando intento una tercera vez con la clave correcta antes de 5 minutos, entonces el sistema me rechaza igual, por bloqueo temporal.

HU04 — Registrar una salida de material. Como técnico, quiero registrar la salida de un material que usé, para que el stock quede actualizado y quede constancia de quién lo retiró. - Dado un material con stock disponible suficiente, cuando registro una salida por esa cantidad, entonces el stock disminuye exactamente en esa cantidad y el movimiento queda registrado con mi usuario. - Dado un material con stock insuficiente, cuando intento registrar una salida mayor al disponible, entonces el sistema la rechaza y el stock no cambia.

HU07 — Filtrar el historial. Como técnico o coordinador, quiero filtrar el historial por material, responsable, tipo o rango de fechas, para auditar rápido sin revisar todo el historial manualmente. - Dado varios movimientos de distintos materiales y usuarios, cuando filtro por un material y un responsable a la vez, entonces solo veo los movimientos que cumplen ambas condiciones.

HU08 — Intentar modificar un movimiento. Como cualquier usuario (incluido administrador), quiero que el sistema impida editar o borrar un movimiento ya registrado, para que el historial sea confiable para una auditoría. - Dado un movimiento ya guardado, cuando intento editarlo o eliminarlo, entonces el sistema lo rechaza siempre, sin excepción de rol.

HU09 — Consultar el dashboard. Como técnico, coordinador o administrador, quiero ver un resumen general del inventario al entrar al sistema, para identificar de un vistazo qué materiales necesitan reposición.

HU11 — Protección contra autobloqueo (administrador). Como administrador, quiero que el sistema me impida desactivar mi propia cuenta, para no dejar el sistema sin ningún administrador activo por error.

3 Arquitectura y stack tecnológico

3.1 Vista general

El sistema se compone de tres procesos independientes, cada uno corriendo por su cuenta (solo la base de datos corre en un contenedor Docker):

Componente	Tecnología	Cómo corre	Puerto
Frontend	Next.js 14 (App Router) + Tailwind CSS	Proceso local (npm run dev:frontend)	3000
Backend	NestJS + TypeORM	Proceso local (npm run dev:backend)	3001
Base de datos	PostgreSQL 16	Contenedor Docker	5432

3.2 Capas y responsabilidades

Capa	Responsabilidad	Ejemplo concreto
Frontend	Mostrar datos, capturar formularios, mantener la sesión (cookie con el JWT)	<code>MaterialForm.tsx</code> valida que el usuario llene los campos y llama al backend
Backend — Controllers	Recibir la petición HTTP, aplicar guards de autenticación/rol	<code>MaterialsController</code> exige <code>JwtAuthGuard</code> en todos sus endpoints
Backend — Services	Aplicar las reglas de negocio reales	<code>InventoryMovementsService.calculateMaterialCost</code> decide cómo cambia el stock según el tipo de movimiento
Backend — TypeORM	Traducir objetos a filas de la base de datos	Entidades <code>Material</code> , <code>InventoryMovement</code> , <code>User</code>
Base de datos	Persistir la información de forma durable	PostgreSQL, con <code>synchronize: true</code> en desarrollo

El frontend nunca habla directo con la base de datos: todo pasa por el backend, que es el único que aplica las reglas de negocio. Esta separación en capas (cliente, servidor y base de datos) sigue el estilo arquitectónico de sistemas distribuidos basados en HTTP descrito por Fielding (2000), donde cliente y servidor están desacoplados y se comunican mediante una interfaz uniforme (en este caso, una API REST).

3.3 Justificación del stack

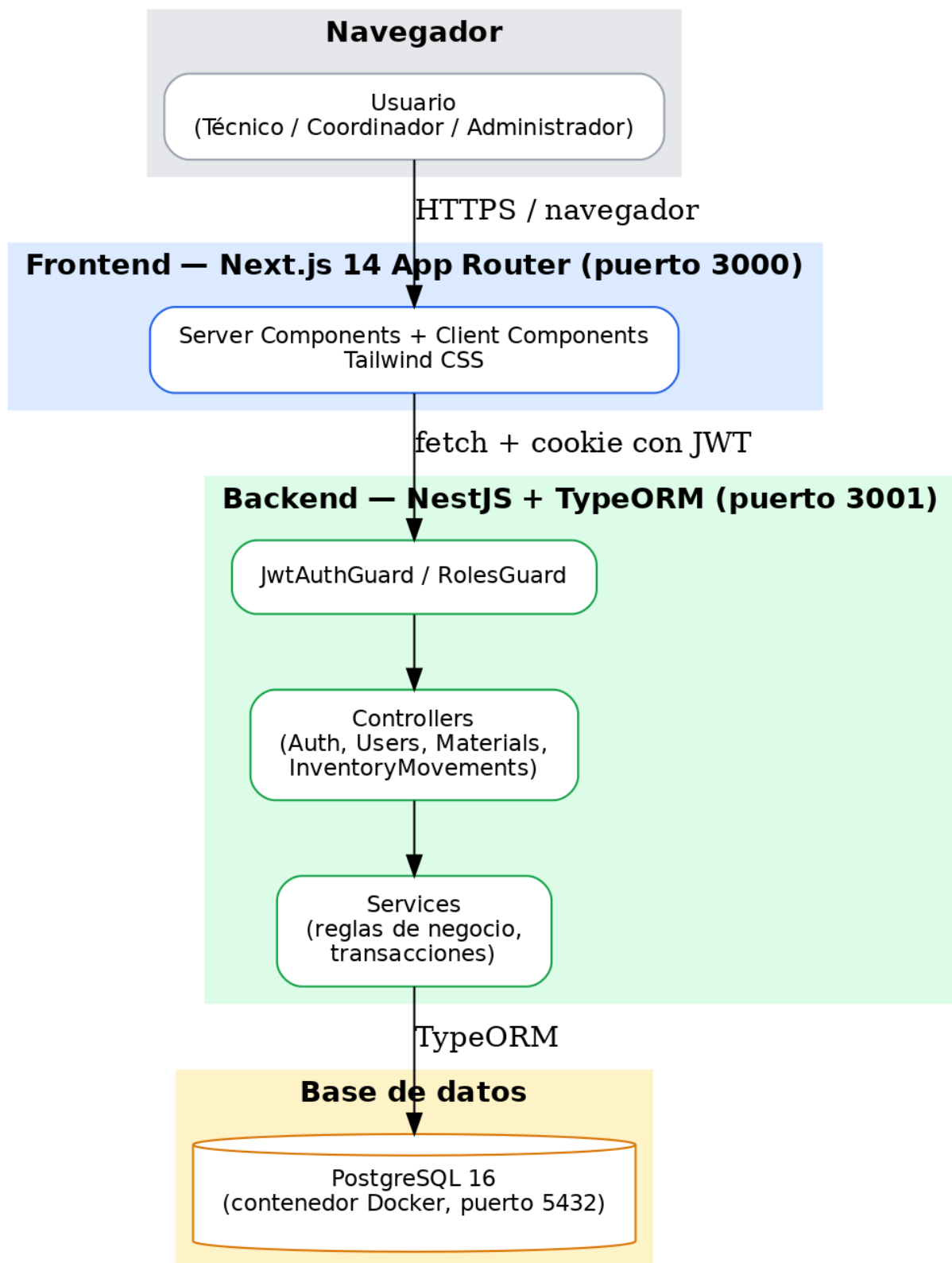


Figura 1: Arquitectura general del sistema

Herramienta	Por qué se eligió
NestJS	Framework de Node.js con estructura modular (controllers, services, módulos) inspirada en Angular, con soporte nativo para inyección de dependencias, guards y pipes de validación — reduce el código repetitivo para aplicar autenticación y reglas de negocio de forma consistente (NestJS, s.f.).
Next.js 14 (App Router)	Permite Server Components que leen la sesión y consultan el backend antes de enviar HTML al navegador, sin exponer lógica sensible en el cliente (Vercel, s.f.).
TypeORM	ORM maduro para TypeScript/PostgreSQL, con soporte de transacciones (<code>DataSource.transaction()</code>), necesario para que los movimientos de inventario nunca dejen el stock a medias.
PostgreSQL 16	Base de datos relacional robusta, con soporte de restricciones de unicidad (usada para code y username) y ejecución en contenedor Docker para reproducibilidad entre equipos.
JWT (JSON Web Token)	Mecanismo de autenticación sin estado, estandarizado en RFC 7519 (Jones et al., 2015): el servidor no necesita guardar sesiones, solo verificar la firma del token en cada petición.
Tailwind CSS	Utilidades CSS aplicadas directamente en el JSX, usado para dar estilo a las 8 pantallas del frontend sin escribir hojas de estilo separadas.

3.4 Autenticación y seguridad

Puntos clave, verificados directamente en el código fuente del backend:

- **Contraseñas:** hash con bcrypt (nunca se guarda texto plano).
- **Bloqueo por intentos fallidos:** 2 intentos seguidos incorrectos bloquean la cuenta por 5 minutos.
- **JWT:** firmado con JWT_SECRET (variable de entorno), expira en 8 horas, siguiendo la estructura estándar de RFC 7519 (Jones et al., 2015).
- **Guards:** JwtAuthGuard protege Materiales, Movimientos de Inventario y Usuarios. RolesGuard + @Roles(ADMIN) protege exclusivamente el módulo de

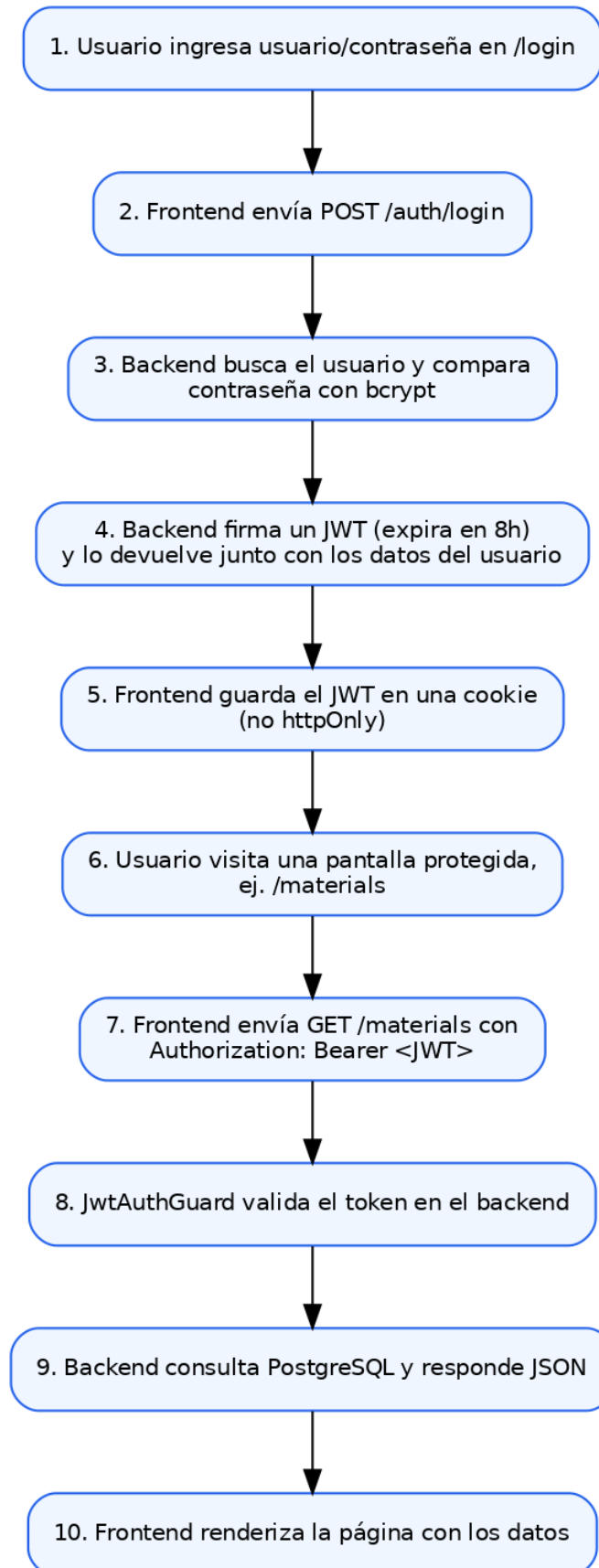


Figura 2: Flujo de autenticación

Usuarios.

- **Sin restricción de rol sobre Materiales/Inventario:** cualquier usuario autenticado (Técnico, Coordinador o Admin) puede operarlos — decisión de alcance documentada, no un descuido.
- **Cookie no-httpOnly en el frontend:** trade-off aceptado para el MVP, necesario porque los Server Components y el middleware de rutas de Next.js necesitan leer la sesión; queda documentado como punto a revisar si el proyecto llega a producción real.

3.5 Patrón de transacciones

Todo movimiento de inventario se ejecuta dentro de una transacción de base de datos, no como dos pasos sueltos. Si algo falla a mitad de camino, no queda ni el movimiento guardado ni el stock a medias.

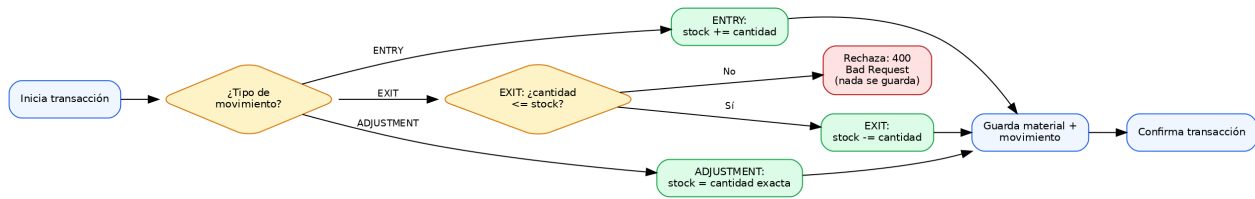


Figura 3: Transacción de un movimiento de inventario

Los movimientos, además, son **inmutables**: no existen endpoints PUT/DELETE para ellos, ni siquiera para el rol Administrador — verificado tanto en el controller (que no expone esas rutas) como en el service (que rechaza esas operaciones si se invocan directamente).

4 Modelo de datos

4.1 Diagrama entidad-relación

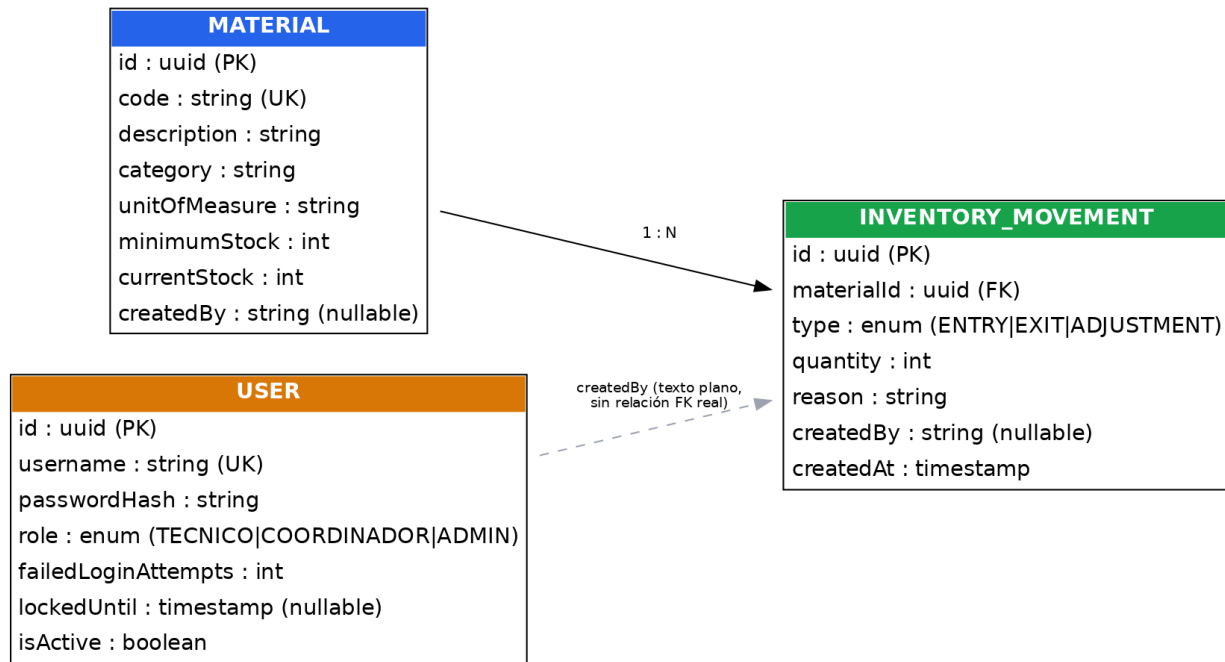


Figura 4: Diagrama entidad-relación (modelo real implementado)

4.2 Entidades principales

MATERIAL — catálogo de repuestos e insumos. code es único (una violación de esta restricción responde 409 Conflict). currentStock nunca se acepta directamente del cliente en creación o edición: solo cambia como efecto de un InventoryMovement.

INVENTORY_MOVEMENT — historial de entradas, salidas y ajustes. Relación N:1 con MATERIAL (cada movimiento pertenece a un solo material). Es inmutable una vez creado.

USER — cuentas del sistema, con rol (TECNICO, COORDINADOR, ADMIN), bloqueo por intentos fallidos y baja lógica (isActive).

4.3 Sobre createdBy (decisión de diseño relevante)

createdBy, presente en MATERIAL e INVENTORY_MOVEMENT, guarda el **username en texto plano** capturado en el momento de creación del registro — no es una relación (FK) formal hacia USER. Por eso, en el diagrama, USER no aparece conectada con una línea sólida hacia las otras dos entidades.

Esta decisión se tomó para evitar joins innecesarios y, sobre todo, el riesgo de exponer accidentalmente passwordHash al popular una relación. Es suficiente para la trazabilidad que pide el sistema (saber qué usuario hizo cada cosa) sin necesidad de

una relación real. La contrapartida aceptada: si un usuario cambiara de username en el futuro, los registros antiguos no se actualizarían retroactivamente (hoy no existe esa función, así que no aplica todavía).

4.4 Discrepancias respecto al modelo original planeado

Antes de iniciar la implementación se había propuesto un modelo más elaborado, con tablas Roles, Categories y Sessions separadas. Durante la construcción se simplificó, y esas diferencias quedan documentadas explícitamente en vez de ocultarse:

Modelo original (planeado)	Modelo real (implementado)	Motivo
Tabla Roles separada, FK desde Users	role es un enum directo en User	Simplificación de MVP — 3 roles fijos no ameritan una tabla aparte.
Tabla Categories separada, FK desde Materials	category es texto libre en Material	Simplificación de MVP, aceptada explícitamente.
Tabla Sessions (id, user_id, token, expires_at)	No existe	La autenticación usa JWT sin estado — el token se valida por firma, no se persiste sesión en base de datos.
quantity/current_stock/minimum_stock con soporte decimal	int en todas las entidades	Diferido — se mantiene entero por ahora; se revisa cuando se agregue una unidad de medida fraccionable real.

5 Funcionalidades desarrolladas

Los cinco módulos entregados, verificados con colecciones de Postman (con tests automáticos) y con pruebas manuales en navegador — no solo revisados en el código:

5.1 Catálogo de materiales

Operaciones CRUD completas (POST, GET con búsqueda parcial por código/descripción, GET por id, PUT, DELETE), más un endpoint de resumen (GET /materials/summary) para el dashboard. code único con manejo explícito del error de duplicado (409, no un 500 genérico).

5.2 Inventario y movimientos

POST /inventory-movements crea un movimiento (ENTRY, EXIT o ADJUSTMENT) dentro de una transacción, actualizando currentStock del material relacionado según la regla de cada tipo. GET /inventory-movements con 5 filtros opcionales y combinables. Sin PUT/DELETE — inmutabilidad por diseño.

5.3 Autenticación y usuarios

POST /auth/login con bloqueo por intentos fallidos. CRUD de usuarios (POST, GET, PATCH, DELETE, y reseteo de contraseña) protegido por @Roles(ADMIN). Política de contraseña validada (mínimo 8 caracteres, mayúscula, minúscula y número). Un administrador no puede autodesactivarse.

5.4 Trazabilidad y auditoría

El mismo endpoint GET /inventory-movements cubre las tres funciones especificadas (historial por material, historial por usuario, filtros por fecha/tipo/responsable) mediante query params combinables — se decidió no duplicar la ruta.

5.5 Consulta y dashboard

GET /materials/summary devuelve indicadores agregados (total de materiales, unidades totales en stock, cantidad y detalle de materiales en stock bajo, definido como currentStock <= minimumStock). El frontend lo consume en la pantalla de inicio (/).

5.6 Pantallas del frontend (8 vistas implementadas)

#	Ruta	Pantalla
1	/login	Iniciar sesión
2	/	Dashboard (indicadores + stock bajo)
3	/materials	Listado y búsqueda de materiales
4	/materials/new	Alta de material
5	/materials/[id]/edit	Edición de material

#	Ruta	Pantalla
6	/inventory-movements	Historial y filtros de movimientos
7	/inventory-movements/new	Registrar movimiento
8	/users	Listado de usuarios (solo lectura, solo ADMIN)

6 Uso de IA como copiloto

El desarrollo de ManteStock se organizó con una división explícita de roles frente a la IA, en vez de usarla como una caja negra que genera código sin supervisión: un asistente (“Claude”, en el rol de arquitectura, planeación, auditoría y documentación) definía checkpoints, escribía prompts precisos, y verificaba el resultado contra el código real antes de aceptarlo como cierto; y un segundo asistente (“Claude Code”, en VS Code) implementaba bajo esos prompts. Esta separación buscaba evitar el riesgo señalado en la literatura sobre productividad con IA: los beneficios de un “AI pair programmer” son reales y medibles (Peng et al., 2023, encontraron que desarrolladores con acceso a GitHub Copilot completaron una tarea 55.8% más rápido que un grupo de control), pero un uso sin supervisión puede introducir errores silenciosos si no se verifica lo que la IA produce o afirma.

6.1 Ejemplos concretos de uso

Generar código bajo prompts precisos. Cada checkpoint (por ejemplo, “007.3 — RolesGuard y CRUD de usuarios admin-only”) se diseñó primero como arquitectura (qué endpoints, qué reglas, qué decisiones) y solo después se tradujo en un prompt específico para que Claude Code lo implementara, siguiendo la metodología Análisis → Diseño → Implementación → Revisión → Validación → Checkpoint definida para el proyecto.

Auditar código para encontrar bugs reales. Al retomar el proyecto en una sesión posterior, en vez de confiar en el resumen de la sesión anterior, se releyó el código real de `MaterialForm.tsx` y se encontró que enviaba un campo (`currentStock`) que el backend rechazaba silenciosamente por la validación `forbidNonWhitelisted: true`, mostrando al usuario un error genérico (“No fue posible guardar el material”) sin explicar la causa real. Se corrigió el frontend y se agregó manejo de errores que expone el mensaje real del backend.

Detectar un patrón de bug repetido. El mismo tipo de error (faltaba `router.refresh()` antes de `router.push()`, dejando que Next.js sirviera datos obsoletos del caché del cliente) se encontró primero en `MaterialForm.tsx/InventoryMovementForm.tsx`, y días después, de forma independiente, en `LoginForm.tsx/LogoutButton.tsx` — reconocer el patrón permitió corregirlo más rápido la segunda vez.

Distinguir un falso positivo de un problema real. Al revisar el estado de Git antes de una entrega, `git status` marcaba casi todo el repositorio como “modificado”. En vez de asumir que había cambios reales pendientes (o peor, subir todo sin revisar), se comparó el contenido línea por línea y se determinó que era ruido de fin de línea (CRLF de Windows vs. LF del repositorio) — evitando un commit masivo e innecesario que habría hecho ilegible el Pull Request para la revisión del profesor.

No corregir datos del usuario sin confirmar. Al cargar un archivo CSV de inventario real, se detectó que una categoría (“Hidráulico”) estaba asignada de forma inconsistente a artículos claramente eléctricos (cables, breakers, transformadores). En vez de “arreglarlo” automáticamente, se le presentó el hallazgo al usuario con la evidencia concreta y se esperó su decisión explícita antes de aplicar el cambio.

6.2 Qué se aceptó, qué se descartó y por qué

Se aceptó el uso de la IA para: generar código repetitivo bajo especificaciones precisas, auditar archivos completos en busca de inconsistencias (más rápido y más exhaustivo que una revisión manual bajo presión de tiempo), y mantener una bitácora de decisiones (CLAUDE.md) actualizada en cada sesión, algo que en la práctica es fácil de descuidar manualmente.

Se descartó aceptar afirmaciones de la IA sin evidencia. La regla de trabajo del proyecto fue explícita: “nunca afirmar que algo está implementado o funciona sin evidencia real” (código visto, resultado de Postman o del navegador). Esto llevó, por ejemplo, a descubrir que un checkpoint que un resumen anterior daba por “en curso” en realidad no existía en el código — se trató como una corrección del estado real, no como avance.

También se descartó dejar que la IA tomara decisiones de arquitectura por su cuenta: cada decisión relevante (por ejemplo, mantener TypeORM en vez de migrar a Prisma, o descartar el módulo de Notificaciones del alcance) se registró explícitamente con fecha y justificación en CLAUDE.md, como una decisión humana documentada, no como algo que “quedó así porque la IA lo hizo así”.

7 Instrucciones de ejecución local

7.1 Requisitos previos

Herramienta	Detalle
Node.js	Versión 20.x (obligatoria — versiones distintas dan problemas de compatibilidad con Next.js 14 en Windows)
Docker Desktop	Para levantar PostgreSQL en contenedor
Git	Para clonar el repositorio

7.2 Pasos

1. **Clonar el repositorio y cambiar a la rama de trabajo:**

```
git clone https://github.com/cmwillamilh-sketch/Proyecto_Inventario.git
cd Proyecto_Inventario
git checkout feature/checkpoints-007-010
```

2. **Configurar variables de entorno:** copiar `.env.example` a `.env` en la raíz del proyecto. Los valores por defecto sirven para levantar el proyecto en un equipo local sin cambios.

3. **Instalar dependencias** (monorepo con npm Workspaces, un solo comando cubre backend y frontend):

```
npm install
```

4. **Levantar la base de datos:**

```
npm run docker:up
```

No hace falta crear tablas manualmente: `synchronize: true` en desarrollo hace que TypeORM las cree automáticamente al arrancar el backend.

5. **Crear el primer usuario administrador** (no existe registro público — es admin-only por diseño):

```
npm run seed:test-user --workspace apps/backend -- admin Admin123! ADMIN
```

6. **Arrancar backend y frontend**, en dos terminales separadas:

```
npm run dev:backend      # http://localhost:3001
npm run dev:frontend     # http://localhost:3000
```

7.3 Verificación

- Backend: `http://localhost:3001/health` debe responder `{ "status": "ok", ... }`.
- Frontend: `http://localhost:3000` debe redirigir a `/login`.

- Iniciar sesión con el usuario administrador creado en el paso 5 debe llevar al dashboard con las tarjetas de indicadores.

8 Conclusiones y trabajo futuro

8.1 Aprendizajes principales

Separar explícitamente el rol de arquitectura/planeación del rol de implementación —incluso trabajando con dos asistentes de IA distintos para cada uno— ayudó a mantener trazabilidad de decisiones y a evitar que el código avanzara sin una razón documentada detrás. La disciplina de verificar cada afirmación contra evidencia real (código, resultados de Postman, comportamiento en el navegador) antes de marcar un checkpoint como terminado evitó al menos un caso concreto de un módulo que se creía “en curso” y en realidad no existía.

Detectar patrones de bugs repetidos (como el de caché del router de Next.js, encontrado en dos parejas de componentes distintas) mostró el valor de una auditoría sistemática de código en vez de solo confiar en pruebas puntuales.

También se aprendió, ya avanzado el proyecto, que la metodología del curso exigía Gitflow (rama + Pull Request + revisión) — descubrir esto a mitad de camino obligó a reorganizar el historial de commits en una rama de trabajo separada en vez de dejarlo directamente en main, una corrección que habría sido más costosa de hacer al final.

8.2 Qué faltó

- **Despliegue en la nube:** fuera de alcance para esta cohorte del seminario (según la guía de entrega); el proyecto corre únicamente en entorno local.
- **Pull Request e integración a main:** el trabajo está en la rama feature/checkpoints-007-010, pendiente de revisión de código antes de integrarse.
- **Gestión de usuarios desde el frontend:** crear, editar, resetear contraseña y desactivar usuarios hoy solo es posible por API (Postman); el frontend tiene una vista de solo lectura.
- **Soporte de stock en unidades decimales/fraccionadas** (kg, litros) — el modelo actual usa enteros.
- **Módulo de Notificaciones:** descartado explícitamente del alcance de esta entrega, no por falta de tiempo sino por decisión consciente de priorización.

8.3 Cómo se abordaría en una siguiente etapa

El orden natural sería: primero cerrar el Pull Request pendiente hacia main con revisión de código, después construir la interfaz de gestión de usuarios (ya que el backend admin-only ya existe y está probado), luego evaluar el paso a un tipo de dato decimal para el stock (probablemente junto con un campo de unidad de medida), y finalmente considerar el despliegue en un entorno accesible fuera de la máquina local, momento en el cual sería obligatorio reemplazar el JWT_SECRET de desarrollo y revisar el uso de una cookie no-httpOnly para la sesión.

9 Componente investigativo

9.1 Marco de referencia

El problema que resuelve ManteStock —la falta de una fuente única de verdad para el control de inventario— está documentado en la literatura sobre administración financiera de inventarios. Durán (2012) señala que el inventario representa una de las inversiones más importantes de una empresa en relación con el resto de sus activos, y que una administración deficiente genera dos riesgos opuestos: inventarios demasiado altos (costos de mantenimiento elevados, recursos inmovilizados) o demasiado bajos (pérdida de clientes, aumento de costos de pedido). Este equilibrio es, en esencia, lo que la función de “stock bajo” del dashboard de ManteStock busca hacer visible de forma automática, en vez de depender del criterio manual de quien lleva la hoja de cálculo.

Desde el punto de vista técnico, la arquitectura cliente-servidor de ManteStock se apoya en el estilo REST (Representational State Transfer), definido originalmente por Fielding (2000) en su tesis doctoral como un conjunto de restricciones para diseñar sistemas distribuidos: interfaz uniforme, comunicación sin estado entre cliente y servidor, y separación de responsabilidades. El backend de NestJS implementa estos principios exponiendo recursos (/materials, /inventory-movements, /users) manipulados mediante los verbos HTTP estándar.

La autenticación sin estado del sistema se basa en JSON Web Token (JWT), estandarizado en el RFC 7519 del Internet Engineering Task Force (Jones et al., 2015). A diferencia de un modelo de sesiones persistidas en base de datos, un JWT contiene la identidad del usuario firmada digitalmente, lo que permite al servidor validar cada petición sin necesidad de una tabla de sesiones — decisión que se refleja directamente en que el modelo de datos de ManteStock no incluye una entidad Sessions.

El backend se construyó sobre NestJS, un framework de Node.js que organiza el código en módulos, controladores y servicios inyectables (NestJS, s.f.), mientras que el frontend usa el App Router de Next.js, que introduce Server Components capaces de ejecutar lógica en el servidor antes de enviar HTML al navegador (Vercel, s.f.) — una decisión relevante para ManteStock, ya que permite leer la sesión del usuario y proteger rutas sin exponer esa lógica al cliente.

Finalmente, el desarrollo del proyecto se apoyó en herramientas de inteligencia artificial como copiloto de programación, una práctica cuyo impacto en la productividad ha sido medido empíricamente: en un experimento controlado, Peng et al. (2023) encontraron que desarrolladores con acceso a un “AI pair programmer” (GitHub Copilot) completaron una tarea de programación un 55.8% más rápido que un grupo de control sin esa herramienta, con beneficios adicionales para desarrolladores menos experimentados. Este hallazgo respalda empíricamente la decisión metodológica del seminario de usar la IA como copiloto explícito del desarrollo, en vez de tratarla como una herramienta opcional o incidental.

9.2 Referencias

Durán, Y. (2012). Administración del inventario: elemento clave para la optimización de las utilidades en las empresas. *Visión Gerencial*, (1), 55-78. <https://www.redalyc.org/pdf/4655/465545892008.pdf>

Fielding, R. T. (2000). *Architectural styles and the design of network-based software architectures* [Tesis doctoral, University of California, Irvine]. https://www.ics.uci.edu/~fielding/pubs/dissertation/rest_arch_style.htm

Jones, M., Bradley, J., & Sakimura, N. (2015). *JSON Web Token (JWT)* (RFC 7519). Internet Engineering Task Force. <https://www.rfc-editor.org/rfc/rfc7519.html>

NestJS. (s.f.). *Documentation*. NestJS. Recuperado el 23 de julio de 2026, de <https://docs.nestjs.com/>

Peng, S., Kalliamvakou, E., Cihon, P., & Demirer, M. (2023). *The impact of AI on developer productivity: Evidence from GitHub Copilot*. arXiv. <https://arxiv.org/abs/2302.06590>

Vercel. (s.f.). *Next.js Docs: App Router*. Next.js. Recuperado el 23 de julio de 2026, de <https://nextjs.org/docs/app>